INGENIERÍA DE SOFTWARE I · USO DEL EQUIPO

Guía para defender el proyecto

Sistema de gestión de solicitudes de soporte técnico

Antes de la defensa

Abran una terminal (PowerShell) en la carpeta `soporte-ti` y ejecuten esto una sola vez:

```
.venv\Scripts\activate
flask --app app iniciar-bd
python -m pytest -v
```

Si al final aparece **27 passed**, todo está listo. Dejen abiertos el informe, la aplicación en vivo y el archivo `app/tickets/reglas.py` en Visual Studio Code.

La idea que deben transmitir: **«Construimos el módulo de tickets: cada ticket sigue estados controlados, cada persona solo hace lo que su rol permite, y 18 casos de prueba encontraron 3 defectos que corregimos en ramas de Git».**

Demostración paso a paso

Usen la aplicación en vivo, que ya trae los tickets de las capturas. La versión local (`flask --app app run` y abrir http://127.0.0.1:5000) tiene las mismas pantallas, pero empieza sin tickets. Contraseña de todos: `Soporte2026*` ; en la pantalla de ingreso basta con pulsar el nombre del usuario.

1. **Entren como Ana (solicitante).** Registren un ticket escribiendo solo espacios en el asunto: aparece «El campo Asunto es obligatorio». Luego regístrenlo bien y muestren el código TK y el estado Nuevo.
2. **Sigan como Ana.** Abran un ticket de Luis (por ejemplo TK-000004): aparece «Acceso no permitido».

3. **Entren como Carla (coordinadora).** Filtren por prioridad Alta y asignen el ticket nuevo a Pedro. Señalen que el técnico inactivo no aparece en la lista.
4. **Entren como Pedro (técnico).** Pasen el ticket a En progreso. Intenten marcarlo Resuelto sin escribir la solución: aparece el aviso. Luego resuélvanlo con una solución.
5. **Vuelvan como Ana.** Cierren el ticket y muestren que ya no hay acciones, pero el historial completo sigue ahí.
6. **Entren como Luis.** Abran TK-000002 (resuelto hace 9 días) e intenten reabrirlo: aparece «El plazo de 5 días para reabrir terminó».

> En la aplicación en vivo, el botón «Restablecer datos de ejemplo» deja todo como al principio.

Comandos para evaluar el comportamiento

Comando	Qué muestra	Qué decir
`python -m pytest -v`	Los 18 casos (27 ejecuciones) en verde.	«Todas las reglas de negocio se comprueban automáticamente.»
`python -m pytest -v -k cp07`	Solo el caso de asignación sin permiso.	«Con -k elegimos un caso por su nombre.»
`python -m pytest -v -k "cp02 or cp13"`	Los casos de textos en blanco y de plazo de reapertura.	«Son pruebas de valores límite.»
`python scripts/medir_rendimiento.py`	Carga 30.000 tickets y termina en «CUMPLE» (tarda menos de un minuto).	«La consulta más lenta tarda menos de medio segundo; el límite es 3.»
`flask --app app respaldar-bd`	Crea una copia en la carpeta `respaldos`.	«Así se cumple el respaldo pedido en RNF-09.»
`git log --graph --oneline --all`	El árbol de ramas y merges.	«Cada funcionalidad y cada defecto tuvo su propia rama.»

Comando	Qué muestra	Qué decir
`git show 6dfb6a9`	El cambio exacto que corrigió DEF-01 (líneas verdes agregadas).	«Esta es la validación de rol que faltaba en el servidor.»
`git show c310d6a`	El cambio de `<` a `<=` del plazo de reapertura.	«Un solo carácter dejaba por fuera el quinto día.»

Para salir de `git show` o `git log` pulsen la tecla **q**.

Cambiar el código para mostrar por qué falla

Si el docente pide demostrar que las pruebas sirven, hagan uno de estos cambios, guarden el archivo y ejecuten el comando. La prueba fallará y explicarán por qué. Todos estos cambios ya se probaron y dan exactamente el resultado indicado.

Para dejar todo como estaba después de cada cambio, ejecuten: `git checkout -- app` y luego `python -m pytest` (debe volver a decir 27 passed).

Quitar el permiso para asignar (DEF-01 / SC-01)

Archivo	`app/tickets/servicio.py`, líneas 113 a 115
Cambio	Poner `#` al inicio de las tres líneas que empiezan con `if not _es_gestor(usuario):`
Comando	`python -m pytest -v -k cp07`
Resultado	**2 failed** con `assert 302 == 403`
Explicación	El servidor esperaba responder 403 (prohibido), pero respondió 302 (aceptó y redirigió): un solicitante logró asignar. Ocultar el botón no protege; el servidor debe revisar el rol.

Cambiar el plazo de reapertura (DEF-03 / RN-09)

Archivo	`app/tickets/reglas.py`, línea 86

Cambio	`return 0 <= dias <= DIAS_REAPERTURA` por `return 0 <= dias < DIAS_REAPERTURA`
Comando	`python -m pytest -v -k cp13`
Resultado	**1 failed, 2 passed**. Falla `[5-True]` con «El plazo de 5 días para reabrir terminó».
Explicación	El día 0 y el día 6 siguen bien, pero el día 5 queda por fuera. Por eso se prueban los valores límite.

Aceptar textos con solo espacios (DEF-02 / RN-02)

Archivo	`app/tickets/reglas.py`, línea 58
Cambio	`valor = valor.strip() if isinstance(valor, str) else ""` por `valor = valor if isinstance(valor, str) else ""`
Comando	`python -m pytest -v -k cp02`
Resultado	**2 failed, 2 passed**: «CHECK constraint failed» y «DID NOT RAISE ErrorValidacion».
Explicación	Sin quitar los espacios, « » parece un texto válido. En el asunto lo detiene la base de datos con un error técnico, y la descripción se guarda en blanco.

Permitir una transición prohibida (RN-05)

Archivo	`app/tickets/reglas.py`, línea 25
Cambio	`ASIGNADO: (EN_PROGRESO,),` por `ASIGNADO: (EN_PROGRESO, RESUELTO),`
Comando	`python -m pytest -v -k cp09`
Resultado	**1 failed** con «DID NOT RAISE ErrorValidacion».
Explicación	Todas las transiciones están en una sola tabla. Si se agrega una que la regla no permite, la prueba lo detecta. También muestra la cohesión: la regla vive en un solo lugar.

Cambiar el largo máximo del asunto (RNF-10)

Archivo	`app/tickets/reglas.py`, línea 34
Cambio	`MAX_ASUNTO = 150` por `MAX_ASUNTO = 100`
Comando	`python -m pytest -v -k cp04`
Resultado	**1 failed** con «El campo Asunto admite máximo 100 caracteres».
Explicación	El asunto de 150 caracteres, que antes era válido, ahora se rechaza. Cambiar un número en las reglas cambia el comportamiento de todo el sistema.

Preguntas que pueden hacer

¿Por qué implementaron Tickets? Es el centro del sistema: reúne casi todos los requisitos (RF-03 a RF-11) y reglas de negocio.

¿Qué es bajo acoplamiento en su proyecto? Que la encuesta de satisfacción (SC-03) se puede agregar como componente nuevo sin cambiar Tickets.

¿Por qué SC-02 es perfectivo y no adaptativo? Adaptativo es cuando cambia el entorno (sistema operativo, leyes). SC-02 es una función nueva para mejorar el servicio.

¿Por qué pytest muestra 27 si son 18 casos? Algunos casos prueban varios datos; por ejemplo, CP-13 prueba los días 0, 5 y 6.

¿Cómo cumplen «sin cambios a medias»? El estado y el historial se guardan en una sola transacción; CP-15 simula un error y el ticket queda igual.

¿Cómo se guarda la contraseña? Cifrada con un hash (scrypt). Se puede comprobar, pero no recuperar.

¿Qué hizo la IA y qué hicieron ustedes? La IA generó la mayor parte del código, las pruebas y la documentación. Nosotros revisamos que cumpliera cada regla, ejecutamos las pruebas, analizamos los fallos y validamos las pantallas. Somos responsables de todo el código.

¿La aplicación en vivo es la misma del repositorio? Es una copia en el navegador con la misma interfaz y las mismas reglas, para mostrarla sin instalar nada. La versión oficial es la de Flask del repositorio, que es la que se prueba con pytest.

Nota para el equipo

> Los tres defectos estaban en la primera versión que se subió a Git, y las pruebas los detectaron de verdad. Sin embargo, esa versión se escribió con esos errores a propósito, para que el ciclo prueba → defecto → corrección quedara registrado. Si el docente pregunta cómo surgieron, díganlo con claridad: son errores muy comunes (validar solo en la pantalla, no quitar espacios, confundir «menor que» con «menor o igual que») y las pruebas se diseñaron para encontrarlos.